STAR + Proxy Metric Learning

Methodology for Remaining Useful Life Prediction

on NASA C-MAPSS Turbofan Engine Datasets

---

1. Problem Statement

Given multivariate time-series sensor data from a turbofan engine, the objective is to predict the Remaining Useful Life (RUL) — the number of operational cycles before failure. RUL prediction is formulated as a regression task with an asymmetric cost function: overestimation of RUL is penalized more severely than underestimation.

We address this problem by learning a structured metric embedding space where samples with similar RUL values cluster together around learnable proxy attractor points. The continuous RUL prediction is obtained as a soft weighted average over the proxy RUL anchors.

2. Data Pipeline

2.1 Dataset: NASA C-MAPSS

The Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) benchmark comprises four sub-datasets (FD001–FD004). Each dataset provides run-to-failure trajectories of turbofan engines with 21 sensor measurements and 3 operational settings per cycle.

Dataset	Operating Regimes	Fault Modes	Train Engines	Test Engines
FD001	1	1 (HPC)	100	100
FD002	6	1 (HPC)	260	259
FD003	1	2 (HPC+Fan)	100	100

2.3 Regime-Specific Normalization

Multi-regime datasets (FD002/FD004) operate under K = 6 distinct conditions. A KMeans clustering is applied on the 3 operational settings to assign each cycle to a regime cluster. Min-max normalization is applied per-cluster using training statistics:

$$\hat{s}_i^{(k)} = \frac{s_i^{(k)} - \min_{train}^{(k)}(s_i)}{\max_{train}^{(k)}(s_i) - \min_{train}^{(k)}(s_i)}$$

where k denotes the regime cluster index and i the sensor index. For single-regime datasets (FD001/FD003), K = 1 reduces this to global min-max normalization.

2.4 RUL Target Construction

For each engine unit, the true RUL at cycle t is computed as the difference between the maximum cycle and the current cycle, clipped at a maximum value:

$$RUL(t) = \min(T_{max} - t,\ RUL_{max}), \quad RUL_{max} = 125$$

This piecewise-linear clipping reflects the practical assumption that degradation is not detectable in the early healthy phase of operation.

2.5 Sliding Window Extraction

Sensor time series are segmented into fixed-length windows using a stride-1 sliding window approach. Given a window size W and a normalized sequence of length T:

- Training: all consecutive windows of length W are extracted with their associated RUL labels.
  labels.
- Testing: only the last W cycles of each engine are used (single prediction per engine).

This produces input tensors of shape (B, W, S) where B is batch size, W is window length, and S = 14 sensors.

3. Model Architecture

The proposed model (STARProxyModel) consists of four stages: (1) dimension-wise patch embedding, (2) multi-scale spatio-temporal attention encoding, (3) multi-scale feature pooling with linear projection into metric space, and (4) proxy-based soft retrieval for RUL prediction.

Learnable positional embeddings are added per-sensor per-patch:

$$\mathbf{h}_{s,j}^{(0)} = \mathbf{e}_{s,j} + \mathbf{PE}_{s,j}, \quad \mathbf{PE} \in \mathbb{R}^{S \times N_{patches} \times d}$$

The resulting tensor has shape (B, S, N_patches, d_model) — a 4D representation that preserves both sensor and temporal structure.

3.2 STAR Attention Block (Spatio-Temporal Attention)

Each STAR block applies two sequential multi-head self-attention (MHA) operations with pre-norm residual connections:

Step 1 — Temporal Attention: for each sensor independently, self-attention is applied across the temporal (patch) dimension. Input is reshaped to (B*S, T, d):

$$\mathbf{h}^{temp} = \text{LayerNorm}\big(\mathbf{h} + \text{MHA}_{temp}(\mathbf{h}, \mathbf{h}, \mathbf{h})\big)$$

$$\mathbf{h}_{out}^{temp} = \text{LayerNorm}\big(\mathbf{h}^{temp} + \text{FFN}_{temp}(\mathbf{h}^{temp})\big)$$

Step 2 — Sensor (Spatial) Attention: for each time patch independently, self-attention is applied across the sensor dimension. Input is reshaped to (B*T, S, d):

$$\mathbf{h}^{sens} = \text{LayerNorm}\big(\mathbf{h}_{out}^{temp} + \text{MHA}_{sens}(\mathbf{h}_{out}^{temp}, \mathbf{h}_{out}^{temp}, \mathbf{h}_{out}^{temp})\big)$$

$$\mathbf{h}_{out}^{sens} = \text{LayerNorm}\big(\mathbf{h}^{sens} + \text{FFN}_{sens}(\mathbf{h}^{sens})\big)$$

Each FFN consists of: Linear(d, d_ffn) -> GELU -> Dropout -> Linear(d_ffn, d).

This factored approach captures intra-sensor temporal dynamics and inter-sensor spatial correlations while maintaining O(S*T^2 + T*S^2) complexity instead of O((S*T)^2).

3.3 Patch Merging (Hierarchical Downsampling)

Between consecutive encoder scales, the temporal resolution is halved via PatchMerging. Adjacent patch pairs along the temporal axis are concatenated and linearly projected back to d_model:

$$\mathbf{h}_j^{merged} = \mathbf{W}_{merge} \cdot [\mathbf{h}_{2j} \| \mathbf{h}_{2j+1}] + \mathbf{b}_{merge}$$

$$\mathbf{W}_{merge} \in \mathbb{R}^{2d \times d}$$

This reduces temporal dimension T -> T/2 at each scale, analogous to pooling in convolutional networks. If T is odd, the last element is dropped before merging.

3.4 Multi-Scale STAR Encoder

The full encoder stacks M scales. Each scale contains L STAR attention blocks, followed by PatchMerging (except the last scale). Features are collected from every scale:

$$\mathbf{F}^{(m)} = \text{STARBlocks}^{(m)}(\mathbf{F}^{(m-1)}), \quad m = 1, \ldots, M$$

The output is a list of multi-scale feature tensors {F^(1), ..., F^(M)} with progressively reduced temporal resolution. This captures both fine-grained and coarse-grained temporal patterns.

3.5 Multi-Scale Feature Pooling

From each scale, the last temporal patch (index -1) is selected and averaged across the sensor dimension, producing a single d-dimensional vector per scale:

$$\mathbf{v}^{(m)} = \frac{1}{S}\sum_{s=1}^{S} \mathbf{F}^{(m)}_{s,\,-1,\,:} \in \mathbb{R}^d$$

All scale vectors are concatenated and linearly projected to the metric embedding space:

$$\mathbf{z} = \mathbf{W}_{proj} \cdot [\mathbf{v}^{(1)} \| \cdots \| \mathbf{v}^{(M)}] + \mathbf{b}_{proj}, \quad \mathbf{W}_{proj} \in \mathbb{R}^{(M \cdot d) \times d_{emb}}$$

4. Proxy-Based Metric Learning for RUL

4.1 Learnable Proxy Attractors

The core novelty lies in the prediction mechanism. Instead of direct regression, K learnable proxy embeddings are maintained, each associated with a fixed RUL anchor value uniformly spanning the RUL range:

$$\mathbf{P} = \{\mathbf{p}_1, \ldots, \mathbf{p}_K\} \in \mathbb{R}^{K \times d_{emb}}, \quad r_k = \frac{(k-1) \cdot RUL_{max}}{K-1}$$

Proxies are initialized orthogonally and learned end-to-end via backpropagation. They serve as virtual attractors that structure the embedding space — samples with similar RUL values should cluster near the corresponding proxy.

4.2 Cosine Similarity and Soft Retrieval

Both sample embeddings and proxy embeddings are L2-normalized to the unit hypersphere. Cosine similarity is computed between each sample and all proxies:

Similarity scores are converted to probability weights via temperature-scaled softmax:

$$w_k = \frac{\exp(\text{sim}(\mathbf{z}, \mathbf{p}_k)/\tau)}{\sum_{j=1}^{K} \exp(\text{sim}(\mathbf{z}, \mathbf{p}_j)/\tau)}$$

where the temperature parameter tau controls the sharpness of the distribution. Lower tau values produce more peaked distributions (harder assignment), while higher values produce softer distributions. Default: tau = 0.02.

The predicted RUL is the weighted sum of proxy anchor values:

$$\widehat{RUL} = \sum_{k=1}^{K} w_k \cdot r_k$$

This formulation enables continuous prediction through discrete proxy anchors, combining the topological structure of metric learning with the smoothness needed for regression.

5. Loss Function: Hybrid Proxy Loss

The training objective combines two complementary losses: a regression loss (MSE) for point-wise accuracy and a distributional loss (KL-divergence) for structured embedding learning.

5.1 Mean Squared Error (MSE) Loss

Standard regression loss between predicted and true RUL:

$$\mathcal{L}_{MSE} = \frac{1}{B} \sum_{i=1}^{B} \left(\widehat{RUL}_i - RUL_i \right)^2$$

5.2 Label Distribution Learning (KL-Divergence)

For each training sample, a target soft label distribution over the K proxies is constructed using Gaussian kernels centered on the true RUL:

$$q_k^* = \frac{\exp\left(-\frac{(RUL - r_k)^2}{2\sigma^2}\right)}{\sum_{j=1}^{K} \exp\left(-\frac{(RUL - r_j)^2}{2\sigma^2}\right)}$$

The bandwidth sigma is set adaptively based on the proxy spacing:

$$\sigma = 0.425 \cdot \Delta r, \quad \Delta r = \frac{RUL_{max}}{K - 1}$$

This calibration ensures that adjacent Gaussian kernels cross at approximately 50% of their peak height, providing smooth overlap without excessive blur.

The KL-divergence loss enforces that the predicted weight distribution matches the target:

$$\mathcal{L}_{KL} = D_{KL}(q^* \| w) = \sum_{k=1}^{K} q_k^* \log \frac{q_k^*}{w_k}$$

5.3 Total Hybrid Loss

The total loss is a weighted combination:

$$\mathcal{L}_{total} = \lambda_{MSE} \cdot \mathcal{L}_{MSE} + \lambda_{KL} \cdot \mathcal{L}_{KL}$$

Default weights: lambda_MSE = 0.05, lambda_KL = 1.0. The dominant KL term structures the embedding space globally, while the MSE term provides direct regression guidance for fine-tuning predictions.

6. Training Procedure

6.1 Optimizer and Schedule

- Optimizer: AdamW with decoupled weight decay.
- Learning rate schedule: CosineAnnealingLR with eta_min = 1e-5.
- Gradient clipping: max_norm = 1.0 to prevent exploding gradients.

6.2 Exponential Moving Average (EMA)

An exponential moving average of the model parameters is maintained throughout training using PyTorch AveragedModel:

$$\theta_{EMA}^{(t)} = \alpha \cdot \theta_{EMA}^{(t-1)} + (1-\alpha) \cdot \theta^{(t)}, \quad \alpha = 0.99$$

The EMA model is used for all evaluation and inference, providing smoother and more stable predictions than the raw training model.

6.3 Dataset-Specific Hyperparameters

Parameter	FD001	FD002	FD003	FD004
Window (W)	32	64	48	64
d_model	64	128	64	128
Heads	1	4	1	4
Scales (M)	2	3	2	3
FFN dim	128	256	128	256
Dropout	0.1	0.3	0.1	0.3
LR	2e-4	2e-5	2e-4	2e-5
Weight decay	1e-4	1e-2	1e-4	1e-2

7. Evaluation Metrics

7.1 Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\widehat{RUL_i} - RUL_i)^2}$$

7.2 NASA Scoring Function (S-Score)

The asymmetric scoring function penalizes late predictions (overestimation) more heavily:

$$S = \sum_{i=1}^{N} s_i$$

$$s_i = e^{-d_i/13} - 1 \quad \text{if } d_i < 0 \quad \text{(early prediction)}$$

$$s_i = e^{\,d_i/10} - 1 \quad \text{if } d_i \geq 0 \quad \text{(late prediction)}$$

where d_i = RUL_predicted - RUL_true. Lower S-Score indicates better predictions with preference for conservative (early) predictions.

8. End-to-End Data Flow Summary

1. Raw sensor data (21 sensors + 3 settings) per engine per cycle.

2. Drop 7 non-informative sensors -> 14 active sensors.

3. KMeans clustering on operational settings (K=1 or K=6).

4. Per-cluster min-max normalization using training statistics.

5. Compute piecewise-linear RUL targets, clipped at 125.

6. Extract sliding windows: X in R^(B x W x 14).

7. Patch embedding: X -> H^(0) in R^(B x S x N_patches x d_model).

8. Multi-scale STAR encoding with PatchMerging: {F^(1), ..., F^(M)}.

9. Last-patch pooling + sensor averaging per scale -> v^(m) in R^d.

10. Concatenate + project: z in R^(d_emb), normalize to unit sphere.

11. Cosine similarity against K=12 L2-normalized proxy attractors.

12. Temperature-scaled softmax (tau=0.02) -> weight distribution w.

13. Predicted RUL = weighted sum of proxy anchor values.

14. Hybrid loss: 0.05 * MSE + 1.0 * KL-divergence.